SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**
Question Count: 5

Code of Conduct

I M.Sandeep(192525095) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sandeep M

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption,

and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Large-Scale Website, Mobile and Transaction Big Data Platform

The described platform handles billions of website clicks, mobile interactions, and customer transactions every day. Such a system would require a distributed big data architecture capable of collecting, storing, processing, and analyzing both structured and semi-structured data.

Probable Distributed Storage System

A distributed file system such as **HDFS** or cloud object storage such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage would probably be used. Data would be distributed across multiple storage nodes rather than stored on a single server.

- Structured transaction data can be stored in relational databases or distributed SQL systems.
- Semi-structured clickstream and mobile logs can be stored as JSON, Avro, or Parquet files in a data lake.
- Data may be partitioned by date, region, customer, or event type.
- Indexing and metadata catalogs would help analysts locate required datasets efficiently.

Data Ingestion

A messaging framework such as **Apache Kafka** would probably collect website events, mobile events, and transaction messages.

The probable flow is:

Website/Mobile/Transaction Sources → Kafka → Data Lake/Data Warehouse → Processing → Analytics

Kafka provides high-throughput event ingestion and allows multiple consumers to process the same data independently.

Batch Processing

Apache Spark would likely be used for large-scale batch processing. It can clean billions of records, join datasets, calculate customer statistics, and generate reports.

For example:

Raw Clickstream → Cleaning → Transformation → Aggregation → Customer Analytics

Older Hadoop environments may also use MapReduce for large batch workloads.

Stream Processing

Real-time analytics could use **Apache Spark Streaming/Structured Streaming**, Apache Flink, or Kafka Streams.

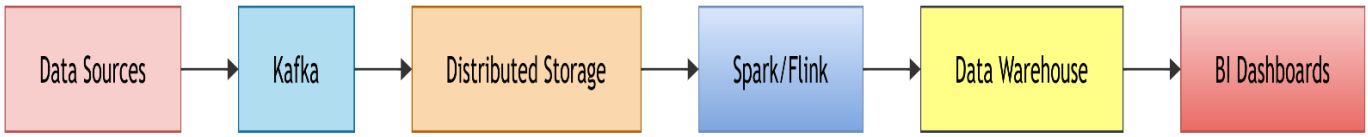
These systems can calculate:

- Real-time website traffic
- Customer activity
- Transaction rates
- Fraud indicators
- Product interactions
- Live business metrics

Business Intelligence and Dashboards

Processed data would be exposed through a data warehouse or analytical database. BI tools such as Power BI, Tableau, or similar dashboard systems could connect to the analytics layer.

Architecture:



2. E-Commerce Personalization and Recommendation Platform

The e-commerce platform continuously collects product views, cart additions, abandoned checkouts, and payment activities. The architecture must support real-time event processing, customer profiling, recommendation generation, and business intelligence.

Event Streaming

Apache Kafka would probably be used as the central event-streaming platform.

Different Kafka topics could represent:

- Product views
- Cart events
- Checkout events
- Payment events
- Search events
- Customer interactions

This separates event types while allowing independent processing.

Customer Profiling Database

A distributed database such as **Cassandra**, MongoDB, or another NoSQL system could store rapidly changing customer profiles.

The profile may contain:

- Customer ID
- Viewed products
- Previous purchases
- Cart history
- Preferred categories
- Location
- Recommendation history

A caching system such as **Redis** could store frequently accessed customer profiles and product information.

Synchronization of Data

The probable workflow is:

User Activity → Kafka → Stream Processing → Customer Profile Update → Recommendation Engine → Website/App

Transaction data from the order database can be combined with clickstream data to create a complete customer behavior profile.

Recommendation Workflow

Machine learning models would analyze customer behavior to predict products that the customer may purchase.

Possible stages are:

1. Collect user events.
2. Clean and transform events.
3. Generate customer and product features.
4. Train recommendation models.
5. Generate product recommendations.
6. Store recommendations in a low-latency database/cache.
7. Display recommendations to customers.

Collaborative filtering, content-based recommendation, or hybrid recommendation models could be used.

Distributed Query Processing

A distributed query engine such as **Presto/Trino**, Spark SQL, or a cloud data warehouse could analyze large clickstream and transaction datasets.

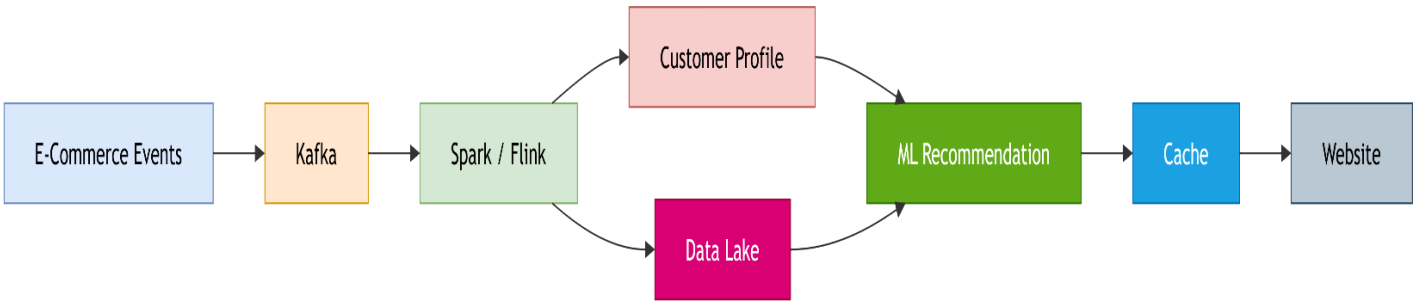
Structured order data could be joined with semi-structured browsing records using customer ID, session ID, or product ID.

KPI and Dashboard Generation

Business intelligence dashboards could display:

- Conversion rate
- Cart abandonment rate
- Revenue
- Average order value
- Product views
- Customer retention
- Regional sales
- Recommendation performance

Architecture:



3. Big Data Healthcare Platform

The healthcare platform processes different types of highly sensitive information, including patient histories, diagnostic images, wearable sensor data, and prescription records. Therefore, a hybrid architecture would be required to manage structured, semi-structured, and unstructured information securely.

Hybrid Database Architecture

Structured information such as patient demographics, prescriptions, appointments, and laboratory results could be stored in relational databases.

Unstructured information such as medical images, scanned documents, and reports could be stored in distributed object storage or a healthcare data lake.

A possible architecture is:

Relational Database + NoSQL Database + Distributed File/Object Storage

Diagnostic images may use specialized medical imaging systems such as PACS and DICOM-compatible storage.

Data Integration

Hospitals, laboratories, pharmacies, and insurance systems would need interoperability mechanisms.

Healthcare standards such as **HL7** and **FHIR** could be used to exchange patient information.

ETL/ELT tools would perform:

1. Data extraction from hospitals.
2. Data transformation and validation.
3. Standardization of formats.
4. Data quality checking.
5. Loading into the central data platform.

Streaming and Machine Learning

Wearable devices generate continuous streams such as heart rate, activity, and other sensor readings.

Privacy and Security

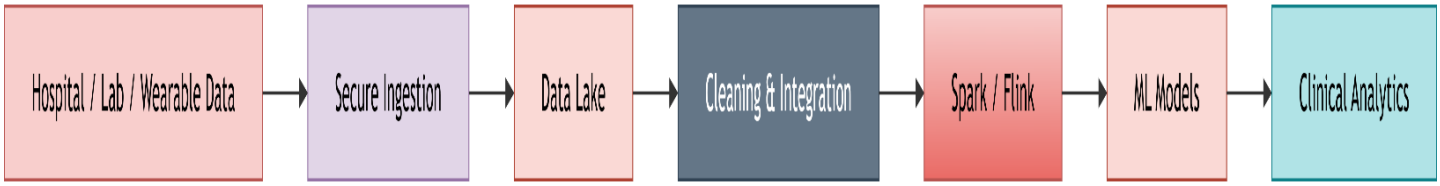
Healthcare data requires strong security controls.

Likely mechanisms include:

- Encryption at rest
- Encryption during transmission
- Role-Based Access Control
- Multi-factor authentication
- Data masking
- Audit logs
- Patient-data access monitoring
- Secure backups
- Data retention policies

The platform would also need to comply with applicable healthcare privacy regulations and organizational policies.

Clinical Analytics Pipeline



The results could support doctors by identifying risk patterns and providing decision-support information.

4. Smart City Traffic and Urban Analytics Platform

A smart city platform continuously receives traffic sensor data, CCTV metadata, weather information, and public transport events. Such a system requires edge computing, real-time streaming, distributed storage, geospatial processing, and visualization.

Edge Computing

Edge devices would be deployed close to traffic cameras, sensors, and transportation systems.

Instead of sending every raw data item to a central server, edge devices can perform initial processing such as:

- Vehicle counting
- Traffic density calculation
- Object detection
- Sensor filtering
- Data compression
- Local anomaly detection

This reduces network bandwidth and response time.

Distributed Messaging

A messaging platform such as **Apache Kafka** would collect events from different city systems.

For example:

Traffic Sensors → Kafka

CCTV Metadata → Kafka

Weather Systems → Kafka

Transport Systems → Kafka

The streams can then be processed independently.

Centralized Storage

A distributed data lake or cloud object storage system could store historical data.

Different storage systems may be used for different workloads:

- Data lake → raw and historical data
- Time-series database → sensor readings
- Spatial database → location-based information
- Data warehouse → aggregated analytical information

Geospatial and Streaming Analytics

Real-time processing engines such as Spark Structured Streaming or Flink could analyze traffic events.

Geospatial operations can determine:

- Traffic density
- Congestion zones
- Average vehicle speed
- Road incidents
- Public transport delays
- Weather-related traffic changes

Machine learning models could predict congestion based on historical and real-time conditions.

Batch and Real-Time Analytics

Real-time analytics provides immediate operational information.

For example:

Sensor → Stream Processor → Congestion Detection → Control Center

Batch analytics analyzes historical information for urban planning.

For example:

Historical Data → Spark → Traffic Pattern Analysis → Infrastructure Planning

Combining both approaches gives city administrators both immediate alerts and long-term planning insights.

Security and Access Control

Because the platform manages sensitive citizen and infrastructure information, it would require:

- Authentication
- Role-Based Access Control
- Encryption
- Network segmentation
- Secure APIs
- Audit logging
- Data anonymization
- Monitoring for unauthorized access

Visualization

Dashboards could provide live maps showing traffic conditions, incidents, weather, and public transport status.

The architecture can be represented as:

Sensors/CCTV/Weather/Transport → Edge Processing → Kafka → Stream Processing → Real-Time Dashboard

and

Historical Data → Distributed Storage → Batch Processing → Urban Planning Dashboard

5. Healthcare Analytics and Predictive Health System

This healthcare analytics system integrates patient records, wearable readings, laboratory reports, and appointment histories. Since the data contains both structured and unstructured information, a hybrid storage and processing architecture is likely.

Hybrid Storage Architecture

Patient records and appointment information are highly structured and can be stored in relational databases.

Wearable readings and medical reports may be stored in NoSQL databases or a distributed data lake.

A probable structure is:

Structured Data → Relational Database

Semi-Structured Data → NoSQL/Data Lake

Unstructured Data → Distributed Object Storage

ETL and Interoperability

The pipeline is:

Hospital/Clinic → ETL → Standardization → Validation → Central Data Platform

Real-Time Monitoring

Wearable devices continuously generate patient measurements. A streaming platform can collect these readings and send them to a real-time processing engine.

For example:

Wearable → Kafka → Spark/Flink → Monitoring System → Alert

An abnormal reading can generate an alert for healthcare professionals.

Distributed Processing

Population-level analysis requires processing large numbers of patient records. Distributed frameworks such as Apache Spark can divide workloads across multiple computing nodes.

They can analyze:

- Disease patterns

- Treatment outcomes
- Patient risk
- Hospital readmission
- Medication effectiveness
- Population health trends

Machine Learning

Machine learning models could calculate patient risk scores based on medical history, laboratory results, wearable measurements, and appointment patterns.

Possible applications include:

- Disease-risk prediction
- Early warning systems
- Readmission prediction
- Treatment recommendation support
- Patient risk classification
- Personalized healthcare

The model output should act as **clinical decision support**, with healthcare professionals making the final decisions.

Overall Architecture